Introduction

Cool. Hello, everyone, and welcome to lecture 4 of CME 295. So today is Friday, October, the 17, which means that the midterm is one week away. So before we start, I'm just going to go over some logistics to make sure we're all aligned on what to expect. So the midterm will take place next week same time. Instead of an hour and 15 minutes, it will be an hour and 30 minutes. So it's 3:30 to 5:00 in this classroom. Just like business as usual. In terms of topics, the midterm will be about lectures 1, 2, and 3, which we had, and this one, which is lecture 4. So just to give you an overview of what you can expect in the midterm, there's going to be some multi-choice questions along with some free form questions, but they're mainly going to be about the things that we've seen in class. So if you watch the recordings, or attend the lectures, and just go through the slides, and know the important formulas, I think you'll be fine. So I know that you may have questions until next week, so that's why after this lecture with Shervin, we will be holding office hours. So feel free to come to us and ask us any questions. And of course, we'll be fully available between now and next week. So in case you have any questions, feel free to ping us on Ed. And yeah, we'll make sure to respond. Cool. I also know that a number of you are auditing this class. So in case you're still interested to take the midterm for some reason, maybe because you have an upcoming interview, just tell us, so that we can just expect the number of copies to print. So we'll be printing this on Monday. So just let us over the weekend in case you're interested. Cool. So that's for the midterm. And then the second piece of news is the final. So we said that we were working on the dates. So we finally finalized the dates which did not change. So it's Wednesday, December the 10. OK. So a little bit late, 7:00 PM to 8:30 PM. So it's a slot that we have. The location is different from this one. So it's in this room. And the final will only cover the second part of the class, which is basically lectures 5 to 9. Any questions on this? Yeah. Oh, yeah. Good question. So is it closed notes? Yes. Yeah. Yes. So the question is, what is the format of the multiple choice? So you'll have-- so we did not finish writing the exam, but it's going to be something like you have a question and let's say, you have three, four possible answers and then you just choose the one that. That's correct. Something like this. And you'll also have some free form, like

you'll have to just answer in your own words. Yeah. Thanks. So the question is, are we allowed to take anything? So it's closed book. So no. Yeah. Nothing. Just a pen. Yeah. Question is no calculator. You will not need a calculator. But speaking of the cheat sheets, so I'm not sure if we mentioned, I think we did, so there is a cheat sheet for this one, which we cannot bring to the exam but you can use just for your studying that's on the website. Class website. So Jess recommends looking at it. Cool. So super clear for everyone? Very cool. Well, OK. As always, we'll be starting the class just recapping what we saw in the previous lecture. So if you remember, we basically studied a new kind of architecture, which was called the mixture of experts, which is such that if you have an input, what you want is to not necessarily activate all the parameters. And so you are in a setting where you have multiple experts, and in the forward pass, you only activate some of them. So that's a sparse MOE. You also have the dense MOE, which basically weights the outputs as a function of the output of the gate. So we saw that this architecture was used in LLMs, and it was mainly used to be able to scale this LLMs without incurring an expensive cost at inference time, because you don't want to activate all the parameters. The second thing that we saw was just defining what an LLM was and in particular, how you could decide on what the next token prediction is. So we saw three methods. First one was we called greedy decoding, which was always taking the highest probable token. The second method we saw was beam search, where we kept track of the K most probable sequences. And then the third one was sampling. So we're not doing the most probable. We're not keeping track of the highest probable sequences. What we do is we sample the next token with respect to the distribution that we get as outputs. And then we saw there's this hyperparameter that's called temperature that allows you to tweak how spiky you want your distribution to like versus not. And we also saw some inference optimization techniques, which are used in practice to avoid having a big cost at decoding time. So I'm not going to just mention everything, but I would say KV cache, for instance, is an important method. So yeah, just recommend just knowing what it is along with the other ones. And with that, we're going to start lecture 4. And actually, I was really looking forward to today because lecture 1, we saw what self-attention was, what a transformer was. Second lecture, we saw some of the tricks that people use today and

some of the variations from the transformer. We introduced what an LLM was last lecture. And this lecture, we're finally going

Pretraining

to see how these LLMs are trained. So today, we're going to focus on LLM training. And the first thing that I'm going to say is if you've been in the ML field for, let's say, more than a few years now, you may have noticed that traditionally, if you had a task, what you would do is train a model specifically for that task. So let's suppose 10 years ago, let's suppose we had a task which was around detecting spam. You would train a model specifically to detect spams. So you would train on the training set, eval on the validation set, and then test on the test set. If you had another use case, let's suppose sentiment extraction, you would train a model specifically for that and so on and so forth. But one could argue that these tasks, they are not completely disjoint. They're all involving just understanding the text. So one could argue we could find a way to somehow leverage the knowledge that we acquired during training for, let's say, one task and reuse that for another task. So this method has a name. It's been around for some time, and it's called transfer learning. So the goal of transfer learning is to not always start from scratch if you have a new task. It's to start with some pre-trained model, and we're going to see what pre-trained is, and then tune it for your task instead of starting from scratch. Well, it's basically the paradigm on which LLMs are trained. So the idea here is that all these tasks, they involve understanding language. So what we're going to do is have what we call a pre-training stage, which involves training your LLM on vast amounts of data to just understand what language, what code is and then have a second stage of, quote unquote, "tuning." And we're going to see a little bit what that tuning is. But in that second stage, we're going to take our pre-trained model and somehow find a way to tune the weights to adapt to a specific task. So as an example, here, we would pre-train a huge model and then suppose for spam detection. We would somehow tune it for that sentiment extraction, same. We would tune it for that and so on. So this is just to take the example from before. And the idea here is in order to obtain these models, we're not going to start from scratch. Cool. So now, we're going to see what pre-training is. So pre-training is by far the most expensive both in terms of

compute, cost, everything part of the training. So what it does is taking a huge amount of data and training your LLM to just predict the next token. And here, by data, what I mean is basically everything you can find. So it can be text in English. It can be text in other languages. It can be even code, can be code in different languages, can be basically the whole internet. We're going to see some of the data sets that people use for that, but you can think of this as just training your model to try to predict anything that's written. And as I mentioned, the objective here is to predict the next token. So if you remember, our LLM is a text-to-text model and most likely a decoder only model in more than 90% of the cases. So what it does is it takes some input text, and it tries to always predict the next token in an iterative basis. So in terms of the data sets that are used, you will see the term Common Crawl a lot on papers. It's basically a data set composed of anything you can find on the internet. So I think they have something like three billion pages per month. So if you go on their website, they have a huge archive. So there's a bunch of other websites as well that you can find in there. So for instance, Wikipedia articles, any social media as well, like Reddit. And there are a lot of Reddit conversations in those data sets. You have a lot of codes, and of course, you have a bunch of places for that. You have GitHub, you have Stack Overflow, all these forums that talk about codes. So all of this is meant for your model to just understand the structure of the language and code. And in terms of size, so it's measured in terms of tokens, number of tokens. And one order of magnitude that I want you to remember is on the order of hundreds of billions or even trillions or even tens of trillions of tokens. So I'll give you an example. So GPT-3 was trained on 300 billion tokens. And for instance, Llama 3, which was I believe published last year, was trained on 15 trillion tokens. So these are huge data sets. So before we go further, I want to introduce two notations.

FLOPs, FLOPS

And I think one of them I introduced it last lecture. The reason why I want to talk to you about these notations is they're used everywhere to talk about how much compute some model needs. So the first notation is FLOPs, which stands for floating operations. And what it is it's a unit of compute. So the higher the FLOPs, the more operations are involved. Because by definition, FLOPs is the

number of operations that involve floating point numbers. So floating point numbers, you can think of them as just numbers with decimal points. So in terms of order of magnitudes, training an LLM is on the order of 10 to the power of 25 FLOPs. And the way you obtain FLOPs, so usually, it's a complicated formula. But in your mind, you can think of it as something that is a function of the size of your data, so the number of tokens that you train it on, and the number of parameters of your model. So there's not an universal formula because it also is a function of the architecture. So you can think, for instance, MOE based LLMs as requiring, let's say, less compute because only some parts are activated compared to, let's say, dense LLMs. But you can just think of it as it's a function of the number of tokens and parameters. It's like O of the product between the two, more or less. And then there's a second notation that I want to introduce, which is also FLOPS, but it's different. So here, FLOPS stands for floating point operations per second. So it's a measure of compute speeds. So it's basically how fast can your hardware execute these operations. And so you also have some order of magnitudes here. But if you're into, let's say GPUs, you will see that in the description of GPUs, they always indicate FLOPS. And we will see that in a second. But I just want to call out that FLOPS here is usually all caps, although you may see some papers that use one for the other, which is confusing. So I'll just recommend just contextualizing this notation with respect to the sentence that it is in because sometimes people actually switch the two. But this is the common notation. So far, so good? Cool.

Scaling laws, Chinchilla law

So now, we know that we have a pre-training step. We know it involves a lot of compute. We know it involves a lot of data. We know our model is large. So what people did was trying to see how the performance evolves as a function of model size and training size. And there is this one paper called Scaling Loss for Neural Language Models that was published in 2020 that performed a bunch of experiments by varying these parameters. And what they found was the more compute you have, the better your model learns about predicting the next token. Same for data set size. So the bigger your training set, the better it is, and the bigger your model, the better it is. So for some time, I think, between 2019 and 2024, you were seeing models that were larger and larger, just people

just building things that were bigger and bigger because according to these experiments, the performance was just getting better. So something else that they noticed was bigger models tend to be more what they call sample efficient. So what that means is for an equal amount of tokens that is processed, you will have a better performance with a bigger model compared to a smaller one. But then you can wonder, we don't have unlimited compute. Compute is expensive. It has a lot of drawbacks. So you have a fixed compute, and people also try to answer the question given a certain amount of compute, how can you fix your training set size and your model size in a way that's more optimal? Because here, you need to decide how big is your model. So what they did is they fixed a unit of compute, which is the color of these curves. And they tried training models of different sizes with different training set size. And what they saw was that there was always a sweet spot here, which followed some kind of relationship, and in particular, this is a table that summarizes, quote unquote, "the optimal set of number of parameters and training set size," which is sometimes called the Chinchilla law. And what they realized was if you have an amount of training set size that's about 20 times the model size, then you're spending your compute in a, quote unquote, "an optimal way." And in particular, GPT-3, for instance, I think it was like 175 billion parameters, if I remember correctly, but it's only trained on 300 billion tokens. So this one, for instance, is, according to this, "really undertrained," quote unquote. I think there was a question. Yeah. [AUDIO OUT] So yeah. The question is, do they fit the neural architecture? So I think by now, everyone agrees that LLMs are transformer-based decoder only models. So everyone uses the same model. So you can assume that when I say LLM here, it basically means decoder only transformer-based models. [AUDIO OUT] Yeah. Question is architecture change does not play a big role. So that's what they say actually in their paper. They say the thing that changes the most is the amount of tokens on which you train and the size of your model. Cool. Any other questions? Yeah. [AUDIO OUT] Oh, yeah. Good question. So question is, is there some kind of transfer learning between different versions of models? So for a lot of these models, they're actually closed source. So they don't exactly reveal these things. But I guess it's an interesting question, one that I cannot answer in a general way. So maybe I think it's the best answer I can give you. But in any case, when you look at some of these papers, they always state how much it costs to train this.

And it's always in the order of millions. It's always an expensive step regardless. Cool. Just speaking of that, so pre-training has a lot of challenges. One of them is cost. So when I say, millions of dollars, it's minimum. I think it can even cost tens of millions of dollars or sometimes hundreds of millions of dollars. It takes a lot of time, and people have been mindful of the impact on the environment. So they've also been including the ecological costs. So the other challenge is that the pre-training step is on data that is up to the time at which you pre-train your model on. So what that means is that the knowledge that you acquire from training on this data set can only go up until the date at which you cut your data set. So this date is called the knowledge cutoff date. And so what that means is your base model, your pre-trained base model has no way to know by itself knowledge that occurred after this date. And speaking of that, a lot of papers, they have tried to edit knowledge, inject knowledge. It's always tricky because there's not a clean way to change the weights in a way that does not penalize some parts. So I guess what people want to do is inject knowledge but not regress in some other domains. And this is a very hard problem. And of course, these models, they try to predict the next token. And there's this question of what if it just generates something that it has seen at training time? So what we call plagiarism. So there's always a risk. So these are all the challenges I just want to illustrate when I said knowledge cutoff date. So if you go on, let's say, the OpenAI website or Google website to look at the model cards, you will always see-- so I'm not sure as you can see from here, but there is always a line on knowledge cutoff date, which tells you on when the pre-training of this model was done. So here, for instance, GPT-5 was released a few weeks ago, and here, it says, the knowledge cutoff date is September 30. So you can guess that they've done their pre-training around that stage. Cool. Any questions on the first part? All good? Perfect. So in this first part, we've seen that pre-training was a crucial step of the LLM training

Training optimizations overview

process. And we've seen all these big numbers. And one could wonder, well, how can you train such a big model on such a big amount of data? How do people do that? So this is what we're going to see here. So this is what I mentioned. So LLMs, you can think of them as decoder only transformer-based

models. So in order to train your model, you need that. You need a lot of data, but then if you look at your architecture, you see that a lot of the operations involve matrix multiplications. And I guess I have a question for you. What is the kind of hardware that loves matrix multiplications? GPUs. Yes. So you also need GPUs. Actually more than one. Yeah. You had a question. [AUDIO OUT] Oh. Question is GPT for inference. So this one, we're going to focus on training, but requirements for GPUs, they differ a little bit between training and inference, but in this part, we're solely focused on training. And speaking of GPUs, I guess, it's not GPUs everywhere because, for instance, Google, they've developed their own hardware that's called TPUs, but any non Google-based models, they've most likely been trained on GPUs. Cool. So in order to train your model, what do you do? So first of all, you have your LLM which is now-- so it is this model, but now, we're representing with a box just for simplicity. You initialize it. It's a lot of parameters. So you can think of the scale as being somewhere around billions to hundreds of billions of parameters, so huge model. And what are the steps involved to train a model? Well, what you're trying to do is to tune the weights, so that the model can learn how to generate the next token. So you have one step called the forward pass, where you have a bunch of data that you're trying to pass through the network. And while we do that, I just want to call out things that are important to note that we need to somehow save in memory. So when you do this forward pass, you have something that's called activations, which are basically the values at each layer that are needed in order to compute the loss. So the loss tells you how off you are compared to the label that you want to train this on. And so the amount of memory that you will use here is dependent on a lot of things. It's dependent on the model size, which impacts the number of activations. It's dependent on how big your batch of data is for training. And it's dependent on how large your context length is, because if you remember, here we have O of N squared complexity because of this self-attention operation, where N is the sequence length. So you have all these parameters that come into play. So once you do the forward pass, let's suppose you compute the loss. You know how off you are compared to your label. Now, the next step is to somehow tweak the weights in a way that minimizes the loss. So how do you do that? There is this another pass called backwards pass. So what this pass does is quantify the direction where the loss is going to be minimized. It's called the gradient. You

take the gradient of the loss with respect to each parameter. Well, gradients, they also need to be saved somewhere in memory. And then you have finally the weights update, which is where you know where the direction at which your loss is going to be minimized. So you apply that update to your weights. And you typically use optimizers out there. Have you heard of Adam optimizer? Yeah. So Adam optimizer is just a fancy version that has some additional quantities, which keep track of-- which are basically a function of the gradients. So you have the first moment and the second moment, which is basically the moving average of the gradient and the squared gradients. And all these quantities, so the first moment, the second moment, you also need to somehow save them somewhere in memory. So it's a lot of things to save. Well, OK. Breaking news, memory is not unlimited. Memory is limited. And so here, what we have in front of us is the description of a GPU. I think. Yeah. H100, which is a very good GPU. And you will see in that description, there's a line on GPU memory. So GPU memory is your amount of memory per GPU. It's 80 gig for this one. It's quite large. So it's in on the order of tens of gigabytes. So you need to store all these things in 80 gigabytes, which is not a lot. So what are we doing? What will you be doing? So I guess the idea is to leverage not one but several GPUs in order to somehow distribute the load across CPUs. And in order to do that, you have several methods, which we will see in a second. So the first set of methods is called data parallelism

Data parallelism with ZeRO

also known as DP. So what this set of methods does is it distributes data across GPUs, so that this forward pass and backward pass, they can all be done independently. And so the idea here is to divide the batch of data across devices, and then in order to do that, of course, you need to have a copy of the model per device. Because of course, you need to compute the activations. You need to compute all these things. But when you do that, you're are able to reduce the memory that is linked to the batch size. So that's called data parallelism. Yes. Question is how about the gradient update? Well, it's a great question. So what do you do when you have independent computations here and there? Well, the gradient is just the average of the gradients for this thing. So you have some communication in between the GPUs that basically

aggregates the gradient for the updates. So I have a question for you. Is this the answer to everything? If we just scale up like this for, I don't know, a lot of GPUs, is it great, always great or do we have cons? Oh, yeah. Great point. So yeah, you have to fit one model. So yeah, this is a great point. So the second point that I will add is you have an additional cost, which is called communication cost, because you need to somehow communicate between your GPUs in order to aggregate some quantities. So your training is going to be slower. It's good. You can scale up the memory. Of course, you need to fit a model on a device, and we will see how we to do that. But you will be incurring those communication costs, so it's not all great. So speaking of the memory and the fact that we want to, I guess, be able to at least store a model per device. So people have realized that there's actually a lot of duplication. And there has been a paper on wanting to deduplicate this duplicate information and this method is called ZeRO, ZeRO redundancy optimization. And the idea is that on each GPU, you store the same parameters, you store the same gradients, you store the same optimizer states. So the idea here is how about we shard, we partition those quantities across GPUs? So the first variation is around sharing the optimizer state. So meaning, we partition those states across the GPUs. So this reduces the memory by a lot. We can also partition the gradients, and we can also partition the parameters. So here, we have no redundant information. Things are just partitioned. Well, the problem is you're going to have even more communication costs, but at least it allows for us to decrease the memory load on each GPU. So this is ZeRO, so there's ZeRO 1, ZeRO 2, ZeRO 3. And I guess the variation that you will choose will be a function of how sensitive you are to, I guess, training time and how big is your model and whether this will be an actual problem, or are you just fine with just storing everything? So that's one set of methods. So this set of methods is, again, data parallelism. So it's basically you having independent sets of data that are handled by different GPUs. Well, you have another set of methods that's

Model parallelism

called model parallelism. So model parallelism tries to parallelize the operation even within one batch. So there's a bunch of methods. I don't want to sound

too like a catalog. So we're not going through them all but one by one. But I will just call out a few that are worth noting. So if you remember last lecture, we talked about MOE-based LLMs and how sequences were being sent to different experts. Well, there is a way to distribute that across GPUs via this expert parallelism techniques, which is having, let's say, one expert on a device, another one on another device. So that's one thing worth noting. Another one, I will say, so a tensor parallelism is when you have big matrix multiplications to somehow cut that in a way that decreases the memory required for that. And maybe the last one I will say is pipeline parallelism. It's when you consider a forward pass as involving several layers. So you're going to say that one GPU is going to only be responsible for, let's say, layers 1 to 3 and then another one layer 3-- sorry, 4 to 5, 4, 5, 6, and so on and so forth. So you also have that kind of parallelism. But anyways, there's a bunch of techniques. And the ones that I mentioned they fall in the bucket of model parallelism. Make sense? No need to know the details on there. But I think just knowing that there are several methods and just a rough idea, I think, is a good thing to have in mind. Cool. So what did we do? So we realized that during the training process, we had to save a lot of things in memory. So what we saw was techniques that reduce the burden of having memory per GPU, so we are trying to distribute that across GPUs. So we saw data parallelism and then the ZeRO method that has some extra optimizations. And we saw model parallelism as well. So now, we're going to see another technique that leverages

Flash Attention

the structure of the GPU. And you may have heard of this technique. It's called flash attention. It was actually developed here at Stanford in 2022. And in order for me to talk to you about this technique, I want to tell you more about what the GPU is composed of. So if you look under the hood-- well, GPU is very complicated and I'm for sure not-- I don't know everything either. But what I know is that we have two kinds of memories in GPU. So you have one kind of memory that's big but relatively slow. That's in the HBM. And then another kind of memory that is fast but much, much smaller, which is on chip next to where the compute happens. That's called the SRAM. So you have HBM and SRAM. HBM has something around tens of gigabytes. So it's like the GPU memory that

you saw in the description. SRAM is much smaller. It's like something around several tens of megabytes let's say. So it's much smaller, but then it is 10 times faster. So this one is a few terabytes per second let's say. And the SRAM is tens of terabytes per second. So it's a noticeable difference in speed. So what we want is to somehow leverage the strength of these kinds of memories in order to speed up the attention computation in an exact way. So what do I mean by exact way? So what I mean is we're not making any approximations to the computation. What we're doing is we're just leveraging the strength of these components and sending the computation in a clever way. So if you remember, the self-attention computation is done with this very important formula. So it's softmax of queries and keys over some scaling factor times V. So this allows queries to interact with everyone else. So in matrix form, you can think of queries as having the number of rows equal to the sequence length. And then columns to being the dimension of the query. And then same for key and value. So you have this big matrix multiplications. So if you do it-- if you do this computation, the standards the vanilla way, what you would do is store them in the big but slow memory component of the GPU. So you would store it in the HBM. So here is what you would do if you were to not do any optimization. So you would take those matrices from the big but slow HBM, perform the computation, and then write it back to the HBM. And then you would read that result again from the HBM, compute the softmax, and then write it back to the HBM. And then you would again load this plus the value matrix, multiply them, and then write them to the HBM. See, there's a lot read and write to the HBM. So it's a lot of data transfer, which actually becomes the bottleneck. So a GPU is very, very fast, but then you spend a lot of time just loading your matrices from the memory. The reason why you do that is because of this softmax operation. So do you remember what a softmax does? So it normalizes the quantities, so that they sum to 1. But it's row dependent meaning that each row needs to sum up to 1. So in a sense, you need that computation to happen first before you do your softmax. If you just look at it like that, you would think, yeah, you need to do the whole thing first. Well, it turns out that you don't need to do everything at once. And this is the core idea behind flash attention. So what flash attention does is it tries to minimize the amount of read and write from and to the HBM and instead takes small blocks-- and it's called tiling, the method is called tiling. It takes small blocks that it

sends to the SRAM, so that it gets computed from end to end before being sent back to the HBM. Does that make sense? So the idea is let's send small matrices into the SRAM, so that it does the whole full end-to-end computation and just send it back to the HBM, because we want to minimize the amount read and write from the HBM. So here is how you would do it. So you remember, the softmax computation with the query and the key and then the value. Well, what you would do is to cut your matrices and then proceed step by step. But then there's a cool trick that I want to talk to you about, which is that you don't need to compute the whole matrix inside a softmax in order to achieve the whole softmax computation. Because if you think about it, let's suppose you have a whole matrix, and then you have different, let's say, columns or submatrices, S1 to SN, well, softmax of this huge matrix is equal to this matrix, where the softmax is taken with respect to each of these submatrices up to some scaling factor. So this is the core trick. And if you want to be convinced of it, just look at the softmax formula, it's exponential of something over some quantity, which is shared across the row. So this scaling factor will just fix this with respect to that. So with this in mind, what we will do is take each respective slices of this matrices, do the whole computation, and then populate the corresponding entry in the output matrix. So we will do that between, let's say, the first slice of the query and the first slice of the keys and the values. And then we will repeat for the other slice until the end. And then we will repeat for the other queries as well until the end. So what the paper explains is how this scaling factor is being computed. So this one is some formula that I did not put on the slide, so it's not necessary for you to memorize the formula. It's just the idea. And the idea is exactly this trick. So once you do that, you basically end up with only one read from the HPM. And these [INAUDIBLE] quantities are stored in the SRAM. And then they're read from the SRAM, which is very fast, and then computed, and then back to the SRAM. And then at the end, in order to accumulate the results, they're being sent back to the HBM. So just to make sure we're clear. So in green, it's basically when it's read from SRAM, and then in blue, it's from the HBM. You have a question? Yeah. [AUDIO OUT] The question is, do you take the whole row or a portion of it? So you can take a portion of it. But just for illustrative purposes, here we take the exclusive. This is just for illustrative purposes. You can think of your matrix as being completely a grid. And then you just multiply accordingly. But

yeah, this is just for illustrative purposes. Yeah. Yeah. So the question is, are you computing alpha and all these quantities on the fly or do you have some estimation? So all of this is exact, and they're computed in an iterative basis. So the way it works is when you populate the output, you will keep on having some extra quantity that will adjust for that. Think of it as just some formula that works. So yeah, I highly recommend looking at the paper. They actually explicit that quite a lot. So the paper is Flash Attention, fast and memory efficient exact attention with IO awareness. All of the links are in the slides. But yeah, I highly recommend just looking at the exact formula in case you want to be convinced. Cool. But the idea makes sense overall? Any questions on this? Cool. Well, this was flash attention, but there's actually another idea from the paper. So this was only the first idea, which was around making the attention computation faster. The second idea is given that the attention computation is faster, now let's try to be smarter about the backwards pass, because when you compute in the backwards pass, when you compute the gradient of the loss with respect to a parameter, the chain rule will surface some activations that you need to have in memory. Well, here, given that computing these activations is very fast, one idea is to just not store activations from the forward pass or at least not store everything, but instead in the backwards pass, compute these activations again. So it's called the recomputation. So when you do the forward pass, you compute an activation to be able to compute the loss, but then instead of saving it to reuse it during the gradient update, you will just discard it and then recompute the activation during the backwards pass with this very fast technique. And when you do that, it's actually quite remarkable. So you do more operations. So gigaFLOPs is just a derivative of FLOPs. So it's the number of operations you're doing. So with flash attention, you're doing more operations because you're actually recomputing things. But then so you also see fewer read and writes from the HBM. So here, for instance, it was 40.3 in the standard way and then 4 point-- so it's like almost a 10x reduction. But then you see that the runtime is also smaller. So this is very remarkable because usually, when you recompute things, you are saving memory but at the expense of runtime. You're just taking longer. But here, you're not only taking less amount of time, you're also saving memory. So you're basically having everything. It's the best of all worlds. So this is what flash attention mentioned. So you will see that there are some other, I guess, variants of flash attention. So

flash attention 2, flash attention 3. And I would say that these are more adaptation of these methods to the current infra, because each new GPU comes with new pros and cons, so strengths and weaknesses. And I guess there's always a way to make these optimizations better. But yeah, I would say flash attention is quite a common trick, and I think it's a very good thing to. So does that make sense? Cool. I'll take that as a yes. OK. Cool. OK, so last thing-- no, we have a few things that I want to talk to you about. So you have your LLM with a bunch of weights.

Quantization

These weights are all floating points. So you can think of them as just being numbers with a bunch of numbers after the decimal. One natural question you can ask yourself is, do you really need to know that much precision after the decimal point to be able to do a good job? I guess put another way, can you just cut your precision in some way to save on memory but then keep the same performance? So this is the idea behind quantization. So quantization is the process of converting the precision of a number from, let's say, one setting to another. In order to better understand that, I think it's important to know how floating points-- sorry, floating point numbers, how they are encoded. So in practice, they're just a bunch of bits. And here, you have some bits that are responsible for, let's say, the exponent, some that are responsible for the mantissa, which is basically how granular your number is. And then one that is about the sign. So you have a bunch of representations of floats. So the most common ones are in this table. So you have so single point precision, half precision, floating point 64, brain float 16 that are each having different granularities for the three dimensions. So if we take the first two rows, which are the two most common representations that you will see out there, floating point 16 is only represented on 16 bits compared to 32. So it takes basically half the memory if you want to think about it this way. But then it's less precise, meaning it's less granular. So I guess one idea that we could have is to somehow decrease the granularity of these weights and these numbers, hoping that it will not impact performance too much. So another thing that I want to mention is back to that GPU description. You see that there is also a bunch of information around the compute speed. So I'm not sure if you can see

very well on there. Can you see very well on the slide? So I'll read it. The compute speed as a function of which representation you're using. So here, if you're using FP64, which is this super granular way of representing numbers, you only have 34 teraFLOPs of compute speed. But then if you're using a, let's say, FP32, which is half of it, you can double your compute speed and so on. You have all these other numbers as well. So the idea is you can save on memory. You can also go faster. So with this in mind, I just want to touch on one last technique before giving it

Mixed precision training

to Shervin, which is mixed precision training. So the idea behind mixed precision training is to leverage different granularities of float representations such that it will not hurt the performance too much but allow you to save on memory and do things faster. So the idea behind mixed precision training is you have your models or you have your model, you keep your weights in high precision, so FP32, and all the operations that you do in the forward and backward pass, you're going to do it in a lower precision. So in this case, FP16. But then the weight updates will be done still in FP32. So the authors of the paper, when they did that, basically what they realized was the performance was not degraded too much. But then you had a lot of savings on memory and then it was running faster. So now you may wonder, well, why are you keeping the weights at a high precision but not the activations and so on? So I can offer you just a bit of intuition. So whenever you perform a forward pass, you're performing that on a set of data. But that set of data can be noisy in itself. So maybe all the decimal points after-- so all the numbers after the decimal point may not be as useful. So you can think of this update as being more in which direction should the weights go to a more optimal state. So it doesn't require them to be super precise, but then it's much more important to keep your weights, so the weights of the model in high precision in order to not accumulate errors due to quantization. That's one way just think about it. But the long story short is if you reduce the granularity of some numbers, then you will have a lot of benefits and not a lot of disadvantages in terms of the performance. Cool. Yeah. That's a great question. So the question is, do you apply this technique to all weights of all layers or just to some of them? So

there has been a lot of papers on that. And the answer is there are some variations to it. And so the answer is not necessarily. So I do have some pointers. Happy to share them with you. But some parts may be more important than others. So this strategy is a little bit a high level idea, but there are always some variations from setup to setup. Yeah. So is your question that people rely on the relationship between compute-- and sorry, not compute, parameters and token from the Chinchilla paper, but it may be different from their setup which may introduce something-- this is basically your question? So the question is, is there also an optimal precision to use? So I would say, people use different things. There's not like a set way where everyone does something. But I would say that these scaling, like this Chinchilla law, is actually something that some authors, they try to reproduce for their own model. So actually, for instance, the Llama 3 paper, there's a whole part around trying to have some relationship between given a fixed amount of compute, what is the optimal number of tokens and then a number of parameters that is unique to their setup. So what people do is they do this thing themselves on a small amount of training set and on smaller models, so that it doesn't cost them too much. They try to see what the relationship is for their setup, and then they extrapolate. And I believe that's how Llama 3-- so I think there's a model that-- so Llama 3, 405 billion parameters. I think they had the whole section just justifying that they came up to this number with some experiments that they run. So to your question, I think it highly depends on the kind of model that someone is training. And they'll probably have this analysis, I guess, with respect to their own setup. Does that answer your question? Cool. Cool. OK. Great. Yeah. The question is, is there something you can do about the range? So there are several types of quantization. We're not going into details here, but I can give you a ZeRO point quantization and absmax quantization, which are different techniques that play with these ranges. There's also quantization technique that Shervin will cover that also talks about that. So yeah, stay tuned for this. But I think ZeRO point quantization and absmax are ones that maybe you can look into for this. Cool. So with that, I'm going to give it to Shervin. Thank you, Afshin. So now, we have seen how a pre-training worked. We're going to see together the next stages of training.

Supervised finetuning

So as we saw with Afshin, pre-training was a way to build the intuition to the model about how language was constructed and about the main characteristics of what is contained in the training data. That is general enough. So this is why we had corpuses that spans huge corpora of texts that are typically of the size of what you find on the internet. So typically, very large and raw and conveying aspects of language. So now, you might want to ask yourself, how can we make such knowledge actually helpful? So this is something that we're going to discuss just in a second. But before we discuss about what could motivate further training, I want to come back to some very simple example that will surface what could be such a need. So let's take our favorite example about our teddy bear. And let's say, we have a very practical use case. So we have thoroughly loved our teddy bear. But now, it has become a bit dirty, so we need to wash it. And you might want to ask your favorite LLM if you can put it in the washer. So with the description that Afshin mentioned of LLM pre-training, I wanted to ask you all what was your opinion about what could the output to this be. So any guesses? Yep. Yeah. Yeah. Excellent. Yeah. So the answer, the guess was maybe another question. So I think that is a great guess that could definitely be it. So here, in the example, we put some other sentence that could be likely, but the gist is the same. It has been trained on next token prediction, not being an assistant or someone that is helpful to you. So this is why it will try to mimic what could be the pattern here and what could be potential likely next words. So for example, the example we put here is something that relates to how the teddy bear is composed of. And because it's in the same domain as teddy bear and then when you talk about washer, then maybe it has been trained on data that includes materials of teddy bears. So that could be it. And I have a question with you all. Are you happy with it? No. Yeah. Exactly. So this is why we're going to see what we can do with the model to make it helpful to you. And this stage is called fine tuning. So we're going to see that it enables us to, in the general case of LLMs, make the LLM a helpful assistance, but also, if you have a specific use case, you can also use this technique to tune the general representation of the language towards your task of interest. But we're going to focus first on what is done for LLMs in general. So first, I want to define some terms. So model fine tuning is commonly known as SFT. SFT is short for supervised fine tuning. So the supervised part suggests that we need

some labels. And it's actually the case. So it's pairs of inputs and outputs that we provide the model to be trained on. And this is why it's called supervised. And then fine tuning is the term that refers to refining the weights that have been already trained. So you start from the pre-trained weights, and you further train the model on additional data to get a fine tuned model. And then one interesting note that I will tell you all is that the objective function, even though it's the same, will differ a bit from the pre-training task. So in the pre-training task, you start with the BOS token, beginning of sentence, and you fit all your corpora of data trying to predict the next token. But here, since we have this supervised fine tuning setup, where you want the model to do something useful in cases of interest, so let's say, when I ask the model about washing my teddy bear, so washing my teddy bear is actually an input that I give to the model. It's fixed. I don't want the model to parrot what I say. I want the model to be a helpful assistance to what I conditioned it on. So in this setup, the input is not a location where you would predict the next token. You would not do teacher forcing on it but rather start from this input and then predict the next token onwards. And then you're going to tune what the optimal distribution of next token can be based on this input. So any questions on the overall idea of SFT before we dive in a bit deeper? Yep. So the question is we don't put a loss over the input. That's right. So you start from the input, and you start predicting the next token. And then the loss calculation starts from there. Everything good? OK. So what I was just telling you is that SFT can be done to tune your own task of interest, but also, it's something that is being done at one of the main stages of training LLMs as you use it every day. And then this transition from the model being a good representation of language to a useful assistance is a subcategory of SFT called instruction tuning. So instruction tuning comes from the fact that we want the model to answer instructions.

Instruction tuning

So actually, just in a bit. First, we're going to look at the data. So with Afshin, we saw that the composition of the data for pre-training was basically the whole internet where you had sources that were carrying knowledge such as Wikipedia. And in general, masses of text of English that tells the model about how English and other languages work, as well as coding. And here, it's slightly

different. For instruction tuning, we're going to look at data that's presents to the model how it can most helpfully respond to instructions. So you can divide this in several categories. So here we present some categories that could be helpful to users of LLMs. For example, story writing. You're interested in writing a story. You give it a set of instructions, and then you have a ground truth that is attached to it. Other examples include a poem creation, list generation, explanation, and many more that are part of the data mixture of SFT. So basically, you run all these supervised fine tuning training with the inputs that is given, and then you train the model on predicting the output. And I'm going now to show what it looks like. So let's say your instruction is here. So you ask your model to do something. So it could be formulated in a different way but just to keep it short, I just said, do X. And then the yellow part is where you start fitting your objective function on. Does this make sense? Great. So as I was mentioning, the data mixture includes instruction geared data. So for example, you had all of these kinds of tasks that users might ask for at inference time. So it's what I put under the category assistant dialogues. And these days, since you have all these large models that already exist out there, and let's say you want to generate a new one, you don't necessarily have to start from scratch. So all of these that I mentioned were originally human written. So you had all these instructions that people gathered, and you had expert linguists that had set of instructions on how to write the best answer that was fluent, helpful, and everything that was geared towards maximizing your happiness as a user. But these days, you can use these already trained LLMs to generate such data. So typically, when you have a large model that is very performant, you can feed it such instructions, sample some generated outputs, and then have a human or some other LLM review the quality. So it's something that speeds up a bit the curation of such data sets. So I just wanted to emphasize on the fact that it's not only human generated these days, but you can have some assistance. And besides all of the categories that you care about, so typically, math or how you write a proof, how things follow each other, or codes, high quality code bases, you also have other aspects that are very important which when you release a product like this to the wider population that come under the umbrella of safety. So you want your model to be helpful but also harmless. So you also have in the data mixture oftentimes a subset that enforces the model's behavior not to repeat some of the harmful

content it might have seen on its pre-training corpus. So it will include techniques that might reject some user prompts. So you might have tested on your own the limits of an LLM today. If you submit queries that might lead to something that is considered bad, it would just say, OK, sorry, I cannot answer this. And this sorry, I cannot answer could be done via regex but very rarely because it's not scalable. And people actually embed this property as part of the model. So you have techniques that rejects user queries that are seen as harmful. Or there is some other phenomenon such as hedging that nuance the output of the model in order to not make blanket statement as well. So you might have data sets that includes such flavors. And you can have many more depending on the task of interest that you're dealing with. But what I'm listing here is what usually the LLMs out there that aim at being general assistance gather. They tend to be like general tasks that are being trained on. Any questions on the data? Yep. Yep. So the question is the story writing example doesn't precise what kind of poetry the story should be about. So it might be ambiguous. How can it generalize? So that is a great point. So we rely on the model's knowledge that it has accumulated at pre-training time to be able to generalize beyond the examples that it has been fed. So let's say, now, you have a story about poetry, and from pre-training, it knows about all kinds of poetry that exist out there. So you could imagine, after fitting such an example at supervised fine tuning time, that you could give more details to this prompt. And then the model has the ability to generalize the generation of the story with respect to those other attributes. So you can have-- I think, the example that you mentioned is a great example of the magical ability of LLMs to adapt to natural language. And all of that has to do with the kind of distribution it has seen in the past and what we teach it. So I think the concept itself of writing stories is the key learning to the model that it can then generalize. Does that make sense? Yeah. OK. Great. I think there was another question here. Yeah. So the question is about the term of alignment. Is this post training what is called as being aligned? So you are ahead of me. In a few slides, we're going to talk about it. Great questions. Any other question? Awesome. So and then just like Afshin did it for the pre-training part, I'm going to give some orders of magnitude. So the same GPT-3 and Llama 3 papers that Afshin referred some numbers from don't actually give their statistics in terms of token numbers for the instruction tuning side but actually in number of examples. So you see you

have about 13 K used GPT-3 and 10 million for Llama 3. And OK. So let's do some quick estimation. Let's say each example is about 1,000 tokens. When you multiply that with the number of examples, you see that the size of the data sets used for SFT is several orders of magnitude lower than the one used for pre-training. So the mental model that you need to have here is that pre-training is a lot of data you want to learn about general characteristics, about language. And then SFT is more exactly what-- so someone was mentioning just now regarding alignments-- was aligning the goal of the model to being suited for your tasks. So typically, very high quality data sets and much more concise and precise in terms of number. So less order of magnitude. OK. Great. So now, let's try this exercise again. What do you think could be the answer to our instruction tuned now model to the same question? Anyone wants to take a guess? You want to try again. Yeah. So the answer was yes. You can put in the teddy bear in the washer, except that you shouldn't put your teddy bear in the washer. You should handwash it. So it will give you the answer that is helpful to the user. And indeed, this time respond to the query. So which is pretty nice. So now, we talked about all the things that are great about supervised fine tuning. Now, I'm going to detail a bit more about what could make it hard or challenging and then motivate the optimization part that we're going to see afterwards. So first, we talked about it a bit. When we have these data sets, these data mixtures for SFT training, we need high quality data. And when you say high quality data, means oftentimes human involved in the loop, you need to make sure it abides with all the rules and all the characteristics that the user will care about. So typically, it's highly involved. So originally, these first models, they were almost all human. But now, you might have some mixture of human and generation. But one good thing about these things is that the data sets they are trained on are reused. So it's a work that you do once, and you can complement with respect to time. But it's still very expensive in time and resources. So the second point I want to mention comes back to a point that someone just made regarding the distribution of your SFT data sets and how it would align with actual inference distribution. So in the case of the story regarding a specific kind of poetry, the distribution of prompts, that precise the kind of poetry seems close enough for generalization to be made. But you could think about examples that widely differ. So let's say you ask for a story that is in some movie plots, and it differs from the stories

that you might see in textbooks. So this could be slightly out of distribution, and it could have trouble generalizing to that. So prompt distribution is very important. And aligning that prompt distribution with respect to the target task is of interest. Yep. Yeah. Great point. So the question is about, let's say, you have SFT tuned your model, and now, you put a training input back to the model. Will you get the same story? So it has to do with the phenomenon that Afshin was mentioning regarding memorization. So in practice, you will not see the same story most likely because the sampling that you're doing is with a non ZeRO temperature. It will maybe have the same flavor but not word for word. Then the flavor of that story, if you have the exact same prompts, might be the same. It depends on what it has seen at pre-training time. But yeah, so the answer is that if I had to give a guess on that precise example, it might be the same flavor but definitely not the same wording. Yeah. So and it might be a different story if the sampling goes through some other region of the space. Stories are highly creative and pre-training corpora of data mixture had all kinds of stories in there. So the specific example of story is likely to generate different outcomes. Yep. So the question is, will how much the model wanders around depend on the temperature parameter? And yes. So typically, when you have a higher temperature, you also-- so it's also called more creative. And it's for that reason. It might also generate things that are less likely in its output distribution with respect to what it has learned. And this is exactly a case of something that we learn. So the answer is yes. So the question is, is there any way to tune that? So when you tune the temperature, you exactly do that. Or were you thinking about something else? Yeah. So the question is, is there something else you could do to have variations in the response? So for that, I have a simple but hard solution. You will need more data in that category that will ground the model into seeing what kind of distribution we target with such queries. And I think that will be one way for the model to generalize more. So I think it will be working on the data part. Was there another question? All good. So yeah. So this is a very relevant question because it touches on the topic of generalization. So yeah. So data mixture matters a lot. And having points that are sparse enough in the distribution space to give the gist to the model of what it needs to learn, rather than repeating the same story again and again, will have a lot to do with generalization powers. And then this is something we are going to see in a second, how do you evaluate such models? The feeling

that you get as a user on how helpful a model tends to be subjective. So how can you put some number on it is going to be a key topic of interest. And then also very soon, we're going to see how to make the computations not that expensive. So Afshin talked about training optimization techniques. But now, when we're at the fine tuning stage, maybe we can go one step further and see what simplification assumption we could make. OK. Awesome. Now, let me dive into one of these challenges, which was the evaluation part. So people have decomposed what they care about in categories. So I'm listing here some dimensions that are evaluated against and that gives some quantitative number. So you have a general language where this benchmark that is popular is generally one score that people reports. It's a MMLU, which massive multitask language understanding. So it has, I think, 50 or so tasks in there where the model is being evaluated against, and you have some score that you can compare with. You have benchmarks on reasoning, on math reasoning, as well as code generation. So here, you have all sorts of acronyms, and there is even way more. So basically, the setting up benchmark has been one area of research. So you always see more and more benchmark coming in because people tend to optimize for those that exist. And then there are some gaps that might be filled by further ones. And the GSM 8K stands for grad school-- so yeah, grad school math. 8K stands for the number of examples. Yeah, I think so. Or maybe G stands for something else, but it's high school topics basically. And one very interesting pattern that people have seen as these models came out and as they were evaluated against these benchmarks is that sometimes for the same kind of model, you see all of a sudden a spike in numbers with respect to some of these benchmarks without a clear explanation behind it. And there is this paper that I highly recommend taking a look, which explores the phenomenon of training on the test task, not the test set. The test task. So when you have benchmarks regarding math reasoning, let's say, it matters a lot if the model has been trained on a data that relates to that kind of reasoning versus not. And this is why when you type in those kinds of benchmarks, you will see that there are sets of so-called auxiliary training that can be used for the model to be trained on the same domain. And in that way, it enables the fact of comparing models with respect to that specific capability. So what the paper tries to convey here is that if you want to compare models between each other, you need to compare the training mixture it has been trained on and then

ensure parity in terms of training on the test task. So you need to make sure that, for example, the two have been trained on the test task or not. And if it's one and the other not, then it might not be a good thing to compare them. It doesn't give you the intrinsic value of the model. So yeah, that's an interesting phenomenon here. Any questions? OK. Great. So we can move on. So one thing that I was mentioning here is that it's very hard to get a sense of how good a model is, even with respect to benchmark scores, because oftentimes what happens is that as they come out, people design data on the training site that resembles more what we try to solve on the benchmark side. So sometimes you end up with models that score great everywhere, but you as a user don't necessarily see added value to it. And it's not the fault of models. It's not the fault of the benchmarks. It's just that it's very hard to give a number that conveys the value to you of the model. So this is why people have come up with other techniques to put a number on model evaluation. And you might have heard of Chatbots Arena. Have any folks heard of it here? Yeah. So it's a website where models can be submitted, and users come in, and they ask their questions, and they're presented responses from two models. And they're being asked to judge which one is better. And then with some pairwise computations done on the website side, they come up with some ranking in the end that ranks model with respect to, quote unquote, "user preference." So it's a number that is being put on the vibes seen by the user. And is it all perfect? Actually, no. So it suffers from several issues that are-- another set of issues that are hard to deal with. So among them, when you have a new model that comes in, you have some noise at the beginning with respect to which other model it's being compared against. And these first few steps actually influence quite a bit the actual ranking, which makes it a brittle property. And there is a paper that actually shows that it's easily possible to rig such a leaderboard. So I'm not sure if there is any evidence that it has been done in the past. But you can use any model, if you ask the question, who are you, it's just going to say who it is. If you ask ChatGPT, who are you. It's going to say, hey, I'm ChatGPT, I'm a helpful assistant. And this paper observed on this very simple property that it was able to detect what model was being evaluated against. And let's say, you have an adversarial player, it could rig the ranking just by selecting the right model. So it's not foolproof. And then on other aspects, so some of the benchmarks that evaluate these models, they're actually curated by experts

that know very well about the target distribution of what a given prompt should give. So it has a set of guidelines that says clearly, OK, this is factual. This is non-factual. So they're able to determine what is good versus bad. And you as a user, you might not know about it. Because let's just take the example of the teddy bear that you wanted to put in the washer, if I don't know that I need to hand wash my teddy bear, and if I have a detailed response regarding you need to wash it with a machine wash cold, and I could, as a user, find these pieces of advice helpful because they are actionable to me. But actually, are they factual? That is a whole other set of issues that you as a user wouldn't be able to tell on a lot of queries. And another challenge that we have here is a user preference. So who here likes it when there are emojis in LLM responses? Yes? No? Yeah. Personally, I do. And I think there are strong opinions. And some people don't like it at all. So they will down rank such responses. But actually, it's something that the user should be able to tell and choose. And then the distribution of people who choose the best model with respect to the distribution of the wider population that is going to use these models is going to be different oftentimes. And I think the emoji case is a good example, because I think generally, emojis are popular in the wider population. But I think domain experts might not like it as much. So I think that mismatch is one that you would see here. And then the last one I will mention here is the safety side. So you as a user, you don't really like it when a model rejects your prompts. When you ask about something, and it says, OK, hey, sorry, I cannot answer it. So there will be a bias towards responses that actually respond to your query, rather than respect some safety principle that might be eventually the intended product decision. So there is also this kind of bias that pops up here. And as I was mentioning, evaluation is a hard problem. So you have all these angles that you can explore. And it's not one number that's going to tell you what you care about. It's the combination of all of these. And in the end, it's tailored to what you actually need. So you would need to see the strengths and weaknesses of a given model and to determine which one corresponds to your use case. So I'm going to come back to one question about alignment here. So we're going to see at the next lecture a further step into aligning the model to do what you want to do in a step called preference tuning. And then the combination of fine tuning and preference tuning, which comes after pre-training, is what we call alignment of the model. So these two steps are called alignments. And I want to

call out one other thing. There is one step we didn't mention here that is called mid training that has been emerging very recently, which consists in a step just after pre-training in aligning the kind of data that the model is being trained on to the tasks that you really care about. So it's the same pre-training objective but aligning the kind of tasks and the kind of data sets to something that you care about. And yeah, so it's an emerging trend. I have not mentioned it, but just so that you know, mid training is something that you would have between pre-training and fine tuning. All good? OK. Great. So now, we're going to tackle one aspect of the challenges that we had mentioned regarding fine tuning, which is computational expense-- the fact of being computationally expensive. And we're going to look at it with one well-known technique

Parameter-efficient finetuning with LoRA

called LoRA, which is a technique to fine tune your weights at the fine tuning stage in an efficient manner. So it's widely used. And it saves a lot of compute. So when you look at your weight matrices, instead of directly fine tuning the whole weight matrix, the LoRA technique decomposes the fine tuning between the weights of the pre-trained model and additional weights that it decomposes into a low rank multiplication. So in this formulation, the pre-trained weights that you have is frozen. So this W0 is frozen. And then B and A are the matrices that you need to tune. So B and A are typically-- so they have dimensions that will match the number of rows and number of columns respectively for the number of rows of B and number of columns of A. But then the dimension of the columns of B and then rows of A is R, which is the rank of these matrices, which is typically taken very small. So the dimensions of W is typically hundreds or thousands, and the dimension of R is typically up to 10 or O of 10. So as you can imagine, it results in much less weights to train. And then I will mention just other techniques to improve efficiency and then ease the fine tuning, some methods called a prefix tuning and adapters. So they're explained in the class textbook. But we're not going to dive deep into them because they're less commonly used. But just so that you know. So let's walk through in detail how does LoRA work. So when you want to fine tune your model, so you have all these ways for which you have already learned a distribution at pre-training time. So one naive way of operating fine tuning would be to

directly iterate on these weights. But what we're doing here, as we said, is to decompose these into the weights that we have already pre-trained on and this product of matrices. And what LoRA says is that you can do a forward pass on both these terms and then adds these quantities in the end. And A and B is going to be a characterization of your task of interest. So let's say you have a model that-- so Afshin was mentioning the kinds of tasks that you can maybe specialize your model on, for example, spam detection. So you would take your pre-trained model, instantiate this B and A in weights, in the weights of your model, fine tune on them. And then what you will have is that B and A are going to be specific to this task of spam detection, similarly for sentiment extraction and so on. So it's a very nice property that you can start from a base model, further tune your weights, and then have your A's and B's that are task specific. So I want to comment now on where were these matrices being learned. So in the original paper of LoRA, it mentioned training only on the attention matrices. But later on, people realized that it might not be the place where it has the most positive impact on performance. And then there is a blog that came out a few weeks ago that studies this into detail, and then they realized that the feed-forward blocks are actually those where putting LoRA is most beneficial. So today, typically, both these components carry LoRA matrices, but the bulk of the performance improvements is actually contained in the feed-forward block. And then I want to mention two interesting properties about LoRA. So when you fine tune with LoRA weights, you want to use a higher learning rate. So typically, 10 times more is the guidance. And then one interesting fact is that it doesn't perform as well when you train it with larger batch size. So I don't have a good theoretical explanation to give you for each of them. It's more empirical observations that have been emitted. But to give the main lines behind the mindset shared from people who study this phenomenon, the first one might be guided by the rank of the LoRA matrices that might be small. And as a result, given the regions of space that it needs to explore, you need a higher learning rate. And then the second one, the hypothesis here is that the training dynamic of products of matrices is different than a full matrix. And this is where the increasing batch size phenomenon occurs. So it's basically the explanations, tentative explanations given there. So now, we're going to explore an optimization of this. But before I dive deep into that part, does anyone have any questions on the LoRA part? Yep. Yep. Great point. So the question is, do

you do grid search on the rank? So you could. So it's a design choice to rank R. Typically, people would have done it before you. So you have an idea of what rank could be well suited for your use case. So you could definitely do grid search, or you could just pick a popular value. I think a 4 is used commonly. You can just go with one of them. Yeah. And we are going to see that the reduction in number of parameters is so huge already that reducing even more maybe doesn't matter that much. That initial reduction by orders of magnitude goes a long way, and that is just a hyperparameter tuning. And you can see a given rank in a given setup as being a design choice. So we have two minutes, and we're going to quickly cover quantized LoRA.

QLoRA

So the techniques that Afshin mentioned just now regarding quantizing weights and so reducing the memory footprint is something that we're going to see just here. So when you look at these matrices W0 and A and B, what people have done in this paper is to quantize the weights of W0 into a format that is very smart and then compute and iterate on these matrices that are being learned, A and B in full precision, which is in that case BF16. And then the quantization of these frozen weights is super smart. It's in a format called NF4, which assumes that the weights are distributed normally. And it splits the space into quantiles rather than buckets of fixed size, which puts about the same amount of number of values into each. So it optimizes the bits that you use for encoding. And what it does is that it does a double quantization process. So it quantizes the weights. And then in a second step, it quantizes the quantization constants that we didn't see at length here. But basically, when you want to convert your full weights in and out of quantized states, you have constants that are generated. And they propose to quantize these constants as well. So this method generated minus 16X VRAM savings, and then the double quantization methods gave some extra savings but not that much. But it's interesting to know. And then we are exactly on time. Thank you for your time.